# Essential Traps for a Minimal RV32 OS

What to handle, what to ignore, and a small dispatcher for a college-project kernel (S-mode, RV32)

## 1. The short list

A minimal OS needs only **three traps working** to run multiple user programs: the timer tick, the system call, and a way to survive a buggy user program. Everything else can go to `panic_trap()` until you need it.

| Priority | Trap | scause (RV32) | Why you need it |
|---|---|---|---|
| **MUST** | Supervisor timer interrupt | 0x80000005 | Preemptive scheduling: the tick that switches processes |
| **MUST** | Environment call from U-mode | 0x00000008 | System calls (write, exit, yield...) |
| **MUST** | Illegal instruction | 0x00000002 | Catches bad user code or a wrong CSR access; also your first test trap |
| **SHOULD** | Load / store / instruction page fault | 0xD / 0xF / 0xC | Needed once Sv32 paging and U-mode are on: a user touching unmapped memory |
| **SHOULD** | Load / store / instruction access fault | 0x5 / 0x7 / 0x1 | Bad physical address (e.g. NULL pointer without paging) |
| Optional | Supervisor external interrupt | 0x80000009 | Interrupt-driven UART or disk; polling works first |
| Optional | Breakpoint (ebreak) | 0x00000003 | Debug aid only |
| Skip | Misaligned load / store / fetch | 0x4 / 0x6 / 0x0 | Just panic or kill the process |
| Skip | Environment call from S-mode | 0x00000009 | Goes to SBI, not your handler |
| Skip | Software interrupt (0x80000001), counter overflow, others | - | Only for multi-core or performance counters |

Legend: MUST = project does not work without it. SHOULD = needed the day you add U-mode and paging. Optional and Skip: send to panic_trap().

## 2. How to handle each essential trap

| Trap | What the handler does | sepc | stval |
|---|---|---|---|
| **Timer** (interrupt, code 5) | 1. Program the next deadline (SBI set_timer). 2. Call the scheduler. 3. Return the frame of the process to run next. | Do **not** change. Resume exactly where interrupted. | 0 (unused) |
| **U-mode ecall** (exception, code 8) | 1. Set frame sepc = sepc + 4. 2. Read syscall number from a7 and arguments from a0-a5. 3. Write the result into the frame's a0. | **Add 4** (ecall is 4 bytes) or the process repeats the ecall forever. | 0 (unused) |
| **Illegal instruction** (code 2) | From U-mode: kill the process, schedule another. From S-mode: kernel bug, panic. | Print it: it is the bad instruction's address. | Instruction bits, or 0 (platform dependent) |
| **Page / access fault** (codes 12, 13, 15, 1, 5, 7) | From U-mode: kill the process. From S-mode: panic. (Later: demand paging for page faults.) | Print it: address of the faulting instruction. | **Faulting virtual address** (or 0). Print it. |

Deciding user vs kernel: read **sstatus.SPP** (bit 8) from the saved frame. SPP = 0 means the trap came from U-mode; SPP = 1 means it came from S-mode (your kernel).

## 3. CSRs you actually touch

| CSR | Minimal use |
|---|---|
| **stvec** | Write the address of trap_entry once at boot, Direct mode (low 2 bits = 0). Address must be 4-byte aligned. |
| **sscratch** | Holds the kernel stack pointer while user code runs; swap with sp at trap entry (needed once you have U-mode). |
| **scause** | Read in trap_dispatch. Bit 31 = interrupt; low bits = code. |
| **sepc** | Saved/restored by the frame. Edit only for ecall (+4). |
| **stval** | Read for diagnostics in panic and kill messages. |
| **sstatus** | SIE (bit 1) = global interrupt enable; SPP (bit 8) = where the trap came from. Saved/restored by the frame. |
| **sie** | Set STIE (bit 5) to allow the timer: `li t0,0x20 ; csrs sie,t0` |
| **satp** | Only when you add Sv32 paging (bit 31 = MODE). |

Enable interrupts last: program the timer first, then set sie.STIE, then sstatus.SIE.

## 4. Timer setup on RV32 (SBI)

| Step | Detail |
|---|---|
| Read time | rdtime and rdtimeh (64-bit counter on RV32) |
| Next deadline | now + interval (on QEMU virt the timebase is typically 10 MHz, so 100000 ticks = 10 ms) |
| Set it | SBI TIME extension: a7 = 0x54494D45, a6 = 0, a0 = low 32 bits, a1 = high 32 bits, then ecall |
| Clearing the interrupt | Setting a new deadline in the future clears the pending timer interrupt, so the handler must always re-arm it |

## 5. Minimal dispatcher (kernel/trap.c)

Compiles with `gcc -std=c11 -Wall -Wextra` with no warnings. Assumes your frame stores x1..x31 at word indices 0..30, then sepc, sstatus, scause, stval at 31..34 (check against your trap.s).

```
#include <stdint.h>

/* Frame word indices (from trap.s): xN is at index N-1, x1..x31 = 0..30 */
#define FR_A0       9      /* x10 */
#define FR_A7       16     /* x17 */
#define FR_SEPC     31
#define FR_SSTATUS  32
#define FR_SCAUSE   33
#define FR_STVAL    34

#define SSTATUS_SPP   (1U << 8)        /* 0 = trap came from U-mode */
#define SCAUSE_IRQ    0x80000000U

extern void      timer_rearm(void);                 /* SBI set_timer     */
extern uint32_t *schedule(uint32_t *cur);           /* returns next frame */
extern void      syscall_dispatch(uint32_t *frame); /* sets frame[FR_A0] */
extern void      proc_kill_current(void);           /* mark process dead */
extern void      panic_trap(uint32_t cause, uint32_t epc, uint32_t tval)
                     __attribute__((noreturn));

uint32_t *trap_dispatch(uint32_t *f)
{
    uint32_t cause = f[FR_SCAUSE];
    uint32_t epc   = f[FR_SEPC];
    uint32_t tval  = f[FR_STVAL];
    int from_user  = !(f[FR_SSTATUS] & SSTATUS_SPP);
    uint32_t code  = cause & ~SCAUSE_IRQ;

    if (cause & SCAUSE_IRQ) {
        if (code == 5) {                /* supervisor timer */
            timer_rearm();
            return schedule(f);         /* pick next process */
        }
        panic_trap(cause, epc, tval);   /* nothing else enabled */
    }

    switch (code) {
    case 8:                             /* ecall from U-mode */
        f[FR_SEPC] = epc + 4;           /* skip the ecall */
        syscall_dispatch(f);
        return f;

    case 2:  case 12: case 13: case 15:  /* illegal insn, page faults */
    case 1:  case 5:  case 7:           /* access faults */
    case 0:  case 4:  case 6:           /* misaligned */
    case 3:                             /* breakpoint */
        if (from_user) {                /* user bug: kill, keep OS alive */
            proc_kill_current();
            return schedule(f);
        }
        /* fall through */
    default:
        panic_trap(cause, epc, tval);   /* kernel bug: never returns */
    }
}
```

## 6. Build and test order

| Step | Do this | How to test the trap | Expected result |
|---|---|---|---|
| 1 | Set stvec; keep interrupts off | Run `.word 0x00000000` (an illegal instruction) in the kernel | panic prints scause = 0x2 and the right sepc |
| 2 | Add timer: program, STIE, then SIE; return the same frame | Print a dot per tick | Dots appear at a steady rate; no crash |
| 3 | Add U-mode and ecall | User program calls write, then exit | Text prints; ecall returns to the next instruction |
| 4 | Kill on user fault | User program executes an illegal instruction or reads a bad address | Process is killed, message shows scause and stval, other processes keep running |
| 5 | Connect schedule() to the process table | Run two user programs that each print a letter | Letters alternate (preemptive multitasking) |

## 7. Common bugs

| Symptom | Likely cause |
|---|---|
| Program repeats the same syscall forever | Forgot sepc = sepc + 4 for ecall |
| Timer fires once then never again | Did not re-arm the timer in the handler |
| Crash right after the first trap returns | Trap frame offsets in trap.s and trap.c disagree, or frame not 16-byte aligned (use 144 bytes) |
| Interrupt storm / hang | Enabled sstatus.SIE before the timer was programmed |
| User fault crashes the whole OS | Did not check SPP; kernel panics instead of killing the process |
| Trap from U-mode corrupts the kernel stack | sscratch / stack switch missing in trap entry |

Facts in this sheet (cause codes, SPP/SIE/STIE bit positions, stvec modes, stval rules) were checked against the RISC-V Privileged Architecture, Supervisor-Level ISA chapter (v1.13). The SBI call numbers and the QEMU timebase come from the SBI specification and typical QEMU virt behaviour; confirm them on your own setup.